Quanta — Technical Overview

The Problem

Standard RAG pipelines break in predictable ways as corpora grow:

Float32 vectors eat RAM. A corpus of 1 million 768-dimensional embeddings requires ~2.86 GB in float32. At 10 million documents it becomes 28.6 GB — exceeding the RAM of most application servers before the rest of the stack gets any memory.

Dense-only retrieval misses keyword matches. Embedding models compress meaning into a fixed-dimensional space. Rare terms, product codes, legal article numbers, and proper nouns that appear infrequently in training data are poorly represented. A query for "Regulation 2016/679 Article 5" may not surface the exact document if its embedding is similar to many other regulatory texts.

No way to exploit known document relationships. A corpus of legal documents has explicit structure: laws cite other laws, court decisions cite statutes, circulars interpret decisions. Dense search has no knowledge of these relationships. A document written in procedural language may rank low for a substantive query even though it is the most relevant document in the corpus given its structural position.

Post-filter recall loss. Many vector databases filter after ANN search: they retrieve the top-K vectors, then discard those that don't match the metadata predicate. With strict filters and large K, a significant fraction of candidates are discarded. The effective K shrinks and recall drops.

Design Decisions

1. TurboQuant Compression

Quanta stores vectors using turbovec's `IdMapIndex`, which quantises float32 vectors to integers of configurable bit-width (1, 2, 4, or 8 bits per dimension). At 4-bit quantisation, the memory formula is:

```
RAM (bytes) = n_vectors × dim × 4 / 8 = n_vectors × dim / 2
```

For 1 million 768-dimensional vectors:

- float32 baseline: `1,000,000 × 768 × 4 = 2,861 MB (~2.86 GB)`
- 4-bit quantised: `1,000,000 × 768 / 2 = 368 MB (~0.36 GB)`

This is an **8× memory reduction** without a separate ANN server. The index lives in-process; there is no network round-trip.

The trade-off: 4-bit quantisation introduces recall loss. This loss is small in practice for high-dimensional vectors (≥256 dimensions) but is corpus- and query-dependent. Use `bit_width=8` when recall is critical and memory allows.

Quanta does not use HNSW or IVF approximate indexing. turbovec's `IdMapIndex` performs exact search over quantised vectors. This means search time scales linearly with corpus size. For corpora up to a few hundred thousand vectors, this is fast enough for online retrieval. For corpora in the tens of millions, consider a pre-filtering step with `allowed_ids` to constrain the search space.

2. Pre-filter Constrained Query

When `filters` are passed to `MultiRetriever.search()`, Quanta resolves them to a set of allowed chunk IDs before touching the vector index:

```
filter_chunks({"topic": "AI", "year": 2024})
    → [chunk IDs that match]
    → QuantaIndex.search(query, k=k, allowed_ids=[...])
```

This is pre-filter search, not post-filter. The ANN search only considers vectors in the allowed set. PostgreSQL uses a GIN index on the `metadata JSONB` column; DuckDB uses `json_extract_string`. Both resolve the filter before the vector search.

The recall impact: pre-filtering a corpus of 100,000 chunks to a subset of 10,000 means the ANN search operates over 10,000 candidates instead of 100,000. If the filter is highly selective, this can reduce recall. The mitigation is to increase `k` relative to the expected filter selectivity.

3. Graph as Candidate Expander

The graph's role is frequently misunderstood. It is not a relevance scorer. The score formula in `MultiRetriever` is:

```
final_score = (dense_weight/n_indexes) × normalised_dense_score
            + bm25_weight × normalised_bm25_score
            + graph_weight × (1 / hop_distance)
```

The first two terms are computed from the content of the document. The third term is computed from the structural position of the document in the graph. These are different kinds of signals and in general are not comparable.

The reason this distinction matters: a graph score of `1/hop_distance` has no relationship to cosine similarity. Mixing them with equal weights would produce unpredictable and potentially harmful ranking behaviour — documents that happen to be 1 hop from a popular seed would outrank semantically relevant documents with no graph connections.

The correct way to think about `graph_weight` is as a small tie-breaker budget, not as a second relevance signal of equal standing. Setting `graph_weight=0.0` makes the graph a pure candidate expander: it widens the pool of documents available for dense + BM25 scoring without affecting scores at all. Setting `graph_weight=0.3` gives graph-adjacent documents a modest boost that can lift them above dense search's noise floor.

The graph surfaces documents you know are structurally related because you built the edges. Dense search surfaces documents that are semantically similar. When a court decision uses different terminology than the statute it cites, dense search will miss the citation relationship. The graph closes that gap — not by scoring, but by inclusion.

Score merge formula (expanded):

```
# Step 1: dense contribution
per_index_weight = dense_weight / n_active_indexes
for each index:
    normalised_hits = min_max_normalise(raw_scores)
    score_map[id] += per_index_weight × normalised_score

# Step 2: BM25 contribution (when query_text provided and TantivyBM25 active)
# All weights are renormalised to sum to 1 when BM25 is active
eff_bm25 = bm25_weight / (dense_weight + bm25_weight + graph_weight)
score_map[id] += eff_bm25 × normalised_bm25_score

# Step 3: graph contribution (candidate expansion + optional score)
for each graph-expanded id:
    score_map[id] += eff_graph × (1 / hop_distance)
    # id was already 0.0 if not found by dense/BM25
```

4. Dual Docstore

Quanta supports two docstore backends with identical async interfaces:

`PostgresDocStore` — production-grade, concurrent, indexed. Uses raw `asyncpg` with a configurable connection pool. The `metadata JSONB` column has a GIN index that makes `filter_chunks()` fast even at large scale. Tables are created idempotently at `init()` time. Suitable for production APIs with concurrent readers and writers.

`DuckDBDocStore` — zero-setup, single-process, embedded. DuckDB is bundled with Quanta's core dependencies. Synchronous DuckDB calls are dispatched through a single-threaded `ThreadPoolExecutor` so they integrate cleanly with asyncio. Suitable for local development, CI, batch indexing pipelines, and single-process applications. Does not support concurrent writes from multiple processes.

The backend is selected explicitly via `DOCSTORE_BACKEND` (or by instantiating the class directly), not auto-detected from available services. Explicit configuration is production-safe: the application fails loudly at startup if the configured backend is unreachable, rather than silently falling back to a different backend.

5. Optional Everything — Null Object Pattern

Every pluggable component has a null implementation:

Component	Active class	Null class	Activation condition
Graph	`Neo4jGraph`	`NullGraph`	`NEO4J_URI` configured
BM25	`TantivyBM25`	`NullBM25`	`BM25_BACKEND=tantivy`

Component	Active class	Null class	Activation condition
Cache	`RedisCache`	`NullCache`	`REDIS_HOST` configured

Null implementations are not stubs; they are production no-ops. `NullGraph` returns an empty list from every method. `NullBM25` silently drops all adds and returns nothing from search. `NullCache` always misses. Their presence means:

- `MultiRetriever` has no conditional branches for optional components. It always calls `self._graph.expand()`, `self._bm25.search()`, and `self.cache.get()`. The null objects handle the "not configured" case.
- You can start with zero external services and add components incrementally without changing application code — only configuration changes.
- Tests run without external services. Every component can be replaced with its null object or a mock.

Component Map

Component	Technology	Optional	Fallback
Vector index	turbovec	No	—
Docstore	asyncpg / DuckDB	No	—
Graph	Neo4j (Bolt v5)	Yes	`NullGraph`
BM25	tantivy-py	Yes	`NullBM25`
Embedding cache	Redis	Yes	`NullCache`
RAG framework	LlamaIndex	Yes	Direct API

Retrieval Pipeline (Step by Step)

Full pipeline with all components active:

1. **Metadata pre-filter** — if `filters` are supplied, `filter_chunks()` is called on the docstore. The result is a set of `allowed_ids`. If the filter matches zero chunks, `search()` returns immediately with an empty list.

2. **Compute effective weights** — if BM25 is active (non-null and `query_text` provided), all three weights are renormalised to sum to 1. Otherwise `dense_weight` and `graph_weight` are used as-is.

3. **Per-index vector search** — for each active `QuantaIndex`, `search()` is called with the query vector and `allowed_ids` (if any). Raw scores are min-max normalised per index. Normalised scores are multiplied by `dense_weight / n_active_indexes` and accumulated into `score_map`.

4. **BM25 search** — if active, `TantivyBM25.search(query_text, k)` is called. BM25 scores are normalised and multiplied by `eff_bm25`, accumulated into `score_map`.

5. **Graph expansion** — the top `graph_seed_k` IDs from `score_map` become seeds. `Neo4jGraph.expand(seeds, hops=graph_hops)` runs a Cypher BFS query returning `(doc_id,`

`1/dist)` pairs for non-seed neighbors. Each returned ID is added to `score_map` with a contribution of `eff_graph × (1/dist)`. IDs already in `score_map` receive an additional boost.

6. **Sort and hydrate** — `score_map` is sorted by descending score; the top `k` IDs are kept. `docstore.get_chunks(top_ids)` retrieves content and metadata in a single query. Results are assembled as `RetrievalResult` dataclasses with `source` tags.

Unconstrained path (no filters): steps 3–6 only. **Dense-only path (`use_graph=False`, no BM25):** step 3 + half of step 6.

Benchmarks

turbovec recall and QPS benchmarks are published in the turbovec repository. Quanta-level end-to-end benchmarks (including docstore hydration and graph expansion latency) have **not yet been benchmarked**.

Memory reduction from 4-bit quantisation is computed analytically: `n × dim × 4 / 8` bytes vs `n × dim × 4` bytes for float32, yielding an 8× reduction. This is confirmed by the demo at examples/turbovec_4bit_quantization_demo.py.

Limitations

Single-process writes (DuckDB). The DuckDB docstore uses a single file connection and a single-threaded executor. Concurrent writes from multiple processes are not supported. Use PostgreSQL for multi-process or multi-worker deployments.

No distributed index. A `QuantaIndex` lives in one process's memory. There is no built-in sharding, replication, or distributed search. For corpora requiring multiple machines, you would need to implement query fan-out and result merging yourself.

Tantivy sweet spot is under ~50K documents. The tantivy-py bindings are well-suited for small to medium corpora. For larger BM25 indexes, a dedicated search engine (Elasticsearch, Typesense) is more appropriate.

Graph must be user-provided. Quanta does not extract relationships from document content. You must supply nodes and edges via `upsert_node()` and `upsert_edge()`. This is a feature — it means your graph encodes domain knowledge rather than noisy auto-extracted relationships — but it requires upfront work.

No built-in embedding model. You must embed your documents and queries before passing vectors to Quanta. This is intentional: embedding model choice is application-specific and changes frequently. Quanta is model-agnostic.

graph_seed_k sensitivity. Too few seeds and the graph expansion misses valuable paths. Too many seeds and distant, loosely-related documents enter the pool. The optimal value depends on graph density and corpus size. There is no automatic tuning.

Linear search time. `QuantaIndex` performs exact search over quantised vectors. Search time is `O(n × dim)`. For corpora above ~500K vectors, use `allowed_ids` filtering or consider an approximate index.

Comparison

Comparison against common alternatives as of the knowledge cutoff. Capabilities may have changed.

Capability	Quanta	FAISS	Qdrant	Chroma	Milvus
Compressed vectors	✅ 4-bit	✅ PQ/SQ	✅	❌	✅
Metadata pre-filter ANN	✅	❌	✅	⚠️ partial	✅
Graph retrieval	✅	❌	❌	❌	❌
BM25 hybrid search	✅	❌	⚠️ sparse	❌	⚠️ sparse
Embedded (no server)	✅	✅	❌	✅	❌
LlamaIndex native	✅	✅	✅	✅	✅

Notes:

- FAISS has no metadata storage or filtering; those must be implemented outside.
- Qdrant and Milvus support sparse vector hybrid search, which approximates BM25 but is not identical.
- Chroma supports metadata filtering but applies it as post-filter by default in some configurations.
- "Embedded" means the service runs in-process without a separate server. Quanta (DuckDB backend), FAISS, and Chroma fit this profile.

Quanta's differentiator is the combination of graph-augmented retrieval with an embedded, compressed vector index. No other library in this table supports graph expansion as a first-class retrieval component.